7/9/25

Task 7 - Implement python generator and decorators

Aim:

To write a Python program to implement python generator and decorators

7.1 Write a Python program that includes a generator function to produce a sequence of numbers. The generators should be able to:

a) Produce sequence of numbers

b) Product a default sequence of numbers starting from 0, ending at 10 and with a step of 1 if no values are provided.

Algorithm:

1. Define Generator function:
2. Initialize Current Value
3. Generate Sequence
4. Get User Input
5. Create Generator Object
6. Print Generated Sequence

7.1 Program:

```
def number_sequence(start, end, step=1):
        current = start
        while current <= end:
            yield current
            current += step
start = int(input("Enter the starting number: "))
end = int(input("Enter the ending number: "))
step = int(input("Enter the step value: "))
```

Output:

```
Enter the starting number: 1
Enter the ending number : 25
Enter the step value : 5
1
6
11
16
21
```

```
sequence_generator = number_sequence(start, end, step)
for number in sequence_generator:
    print(number)
```

Produce a default sequence of numbers:

Algorithm:

1. Start function
2. Initialize Counter
3. Generate Values
4. Create Generator object
5. Iterate and Print Values

b) Program:

```
def my_generator(n):
    value = 0
    while value < n:
        yield value
        value += 1
for value in my_generator(3):
    print(value)
```

7.2 Imagine you are working on a messaging application that needs to format messages differently based on the user's preferences. Users can choose to have their messages automatically converted to uppercase or to lowercase. Write a program to implement it.

Algorithm:

1. Create Decorators
2. Define Functions
3. Define greet function
4. Execute the Program

Output:

0
1
2

Output:

HI, I AM CREATED BY A FUNCTION PASSED AS AN ARGUMENT

hi, i am created by a function passed as an argum

Program:

```
def uppercase_decorator(func):
    def wrapper(text):
        return func(text).upper()
    return wrapper
def lowercase_decorator(func):
    def wrapper(text):
        return func(text).lower()
    return wrapper
@uppercase_decorator
def shout(text):
    return text
@lowercase_decorator
def whisper(text):
    return text
def greet(func):
    greeting = func("Hi, I am created by a function passed as an argument.")
    print(greeting)
greet(shout)
greet(whisper)
```

VEL TECH - CSE	
EX NO.	7
PERFORMANCE (5)	5
RESULT AND ANALYSIS (3)	3
VIVA VOCE (3)	3
RECORD (4)	4
TOTAL (15)	
SIGN WITH DATE	15

Result:

Thus the program to implement python generator and decorators was successfully executed and the output was verified.